Title: Command Line Basics

Summary: The command line is an important tool for accessing certain programs, platforms, and tools.

Section 1: Why Does the Command Line Matter?

The command line—accessed through the Command Prompt on Windows machines and the Terminal on Mac computers—is a text based interface to access files and programs on your computer.

Early computers ran entirely on text-based prompts, but a turn to graphic user interfaces (GUI), popularized by the early Macintosh computers in the late 1970s and early 1980s, shifted how most users interacted with these machines. GUI refers to the kind of interface that responds to inputs by the mouse, and can be navigated through non-textual inputs.

Think about how you might access a file in your documents folder: you might click on the Finder, if you are using a mac, and then click on the tab on the left hand side of the finder, and then click on the file itself. By clicking through these folders you are interacting with files in a structured hierarchy that can be accessed through the command line as well.

That document on a Mac computer might be located here:

/Users/[username]/documents/[filename]

For Windows users, the file would be located like so:

\Users\[username]\documents\[filename]

What the graphic user interface does is that it navigates and operates the text-based and code-based operations in the computer. It lets users go through the computer in a way that makes sense for those who are less tech savvy.

The GUI is great and has made computers a lot easier for beginning users. It, however, has some limitations when we want to do code-based research. Often code tutorials assume that basic aptitudes, like navigating the command line, installing dependencies, or running prompts within the command line. This introduction to the command line will outline some skills for digital novices, that will be applicable for a range of programs and digital methods.

This tutorial is split for Mac users and Windows users because some of the commands are slightly different for these interfaces.

Section 2a: Command Line on Mac

Section 2a.1: What is the Terminal?

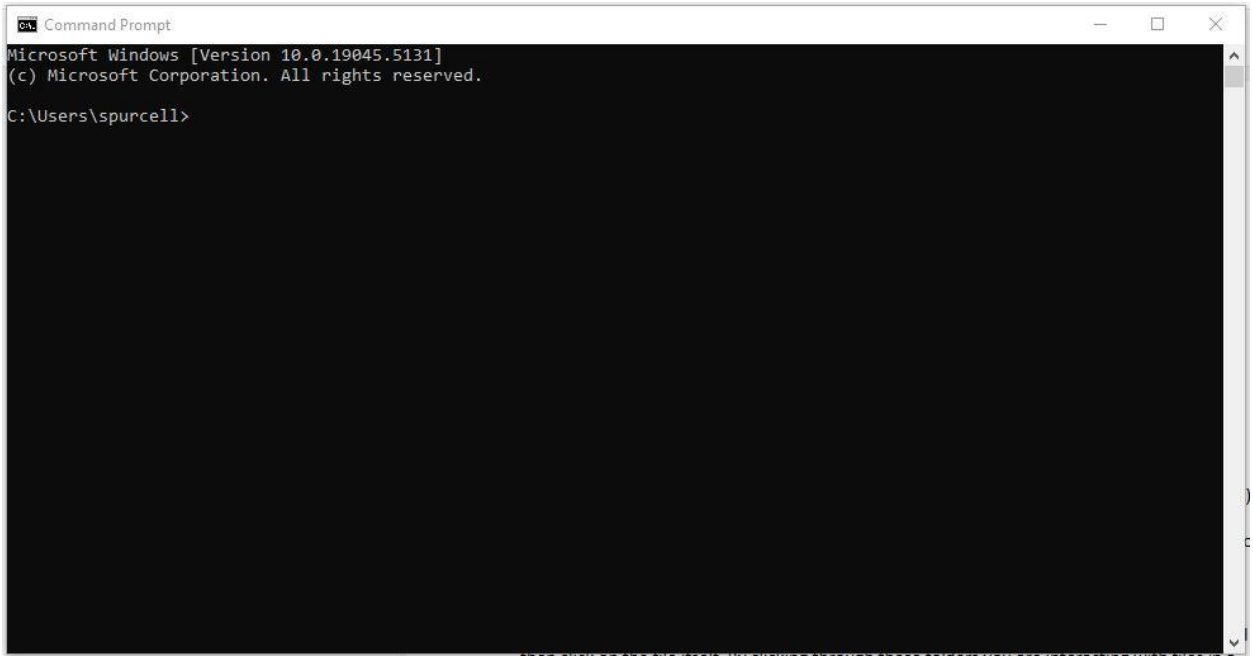
Section 21.2: Navigating your Computer's Folders

Section 2b: Command Line on Windows

Section 2b.1: What is the Command Prompt

Required: You will need to download and unzip the command prompt files for this lesson.

To open the command line on Windows, you will need to open the application called Command Prompt. When you do, you'll find yourself looking at a pretty basic interface.



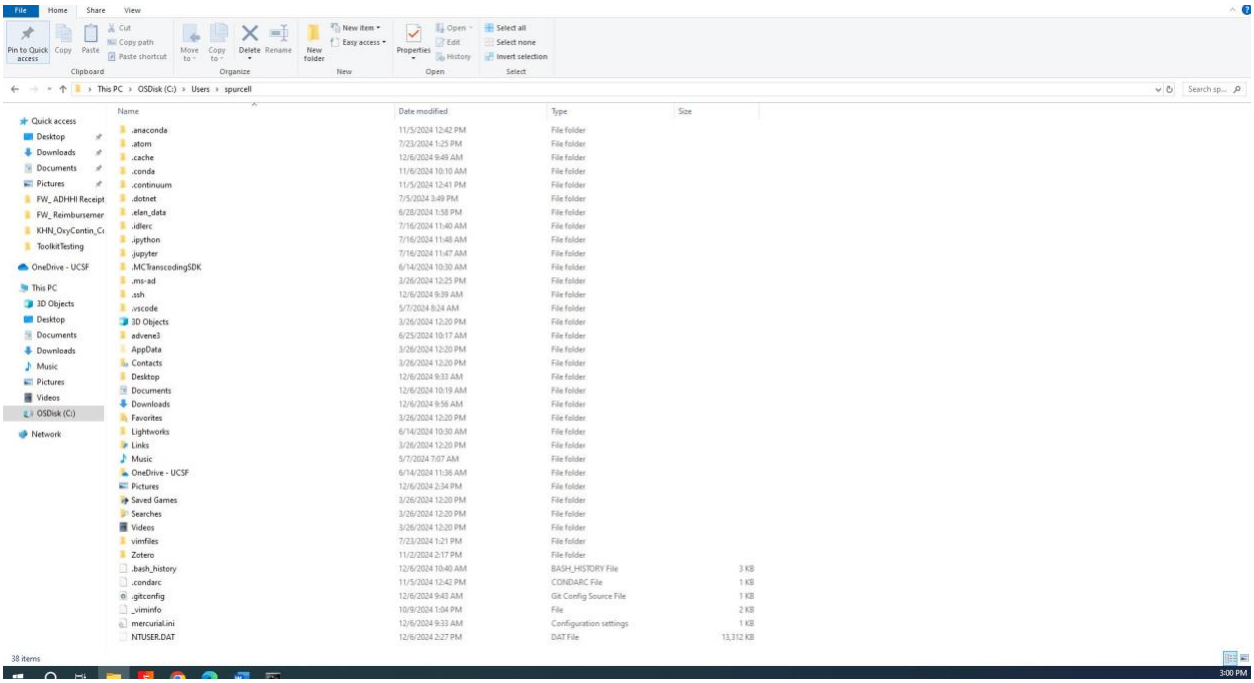
The only way to interact with this interface is with typed commands. The command line is very simple, and it only knows how to run functions that are specifically formatted for it. So if I type in something like "go to documents" it will give me an error code: "'go' is not recognized as an internal or external command, operable program or batch file."

One thing to remember whenever we're working in the command line (or working with code) is that these processes only work if they are given a specific set of inputs with which the program is designed to operate. This means that if you want to use the command line, you have to use a set of predetermined inputs that will output specific results.

Section 2b.2: Navigating your Computer's Folders

To do the next step, please download and unzip the working files folder. (For the examples, I have unzipped this folder in my downloads folder.)

This introduction assumes that the files you have downloaded are in the downloads folder. If they aren't be sure to put them there!



If you are lost, you can always use the GUI folder hierarchy to help you search, because they are looking at the same thing.

Let's write our first command. Type the following into the Command Prompt:

```
dir
```

This command—'dir'—is short for 'directory'. It will output all of the potential directories and files that you can access from your current place in the computer.

> **Important Term: Directory** – A directory is a collection of files and directories which the command line accesses. Often, we call them folders, largely because of their iconography in contemporary GUIs.

The 'dir' command can help us if we are lost, or if we do not know what directories of files we have access to at a given moment.

Let’s look at that output from that command. This is how it looks like on my computer:

```
 Directory of C:\Users\spurcell

12/06/2024  02:25 PM    <DIR>          .
12/06/2024  02:25 PM    <DIR>          ..
11/05/2024  12:42 PM    <DIR>          .anaconda
07/23/2024  12:25 PM    <DIR>          .atom
12/06/2024  10:40 AM             2,743 .bash_history
12/06/2024  09:49 AM    <DIR>          .cache
11/06/2024  10:10 AM    <DIR>          .conda
11/05/2024  12:42 PM                25 .condarc
11/05/2024  12:41 PM    <DIR>          .continuum
07/05/2024  02:49 PM    <DIR>          .dotnet
06/28/2024  12:58 PM    <DIR>          .elan_data
12/06/2024  09:43 AM               234 .gitconfig
07/16/2024  10:40 AM    <DIR>          .idlerc
07/16/2024  10:48 AM    <DIR>          .ipython
07/16/2024  10:47 AM    <DIR>          .jupyter
06/14/2024  09:30 AM    <DIR>          .MCTranscodingSDK
03/26/2024  11:25 AM    <DIR>          .ms-ad
12/06/2024  09:39 AM    <DIR>          .ssh
05/07/2024  07:24 AM    <DIR>          .vscode
03/26/2024  11:20 AM    <DIR>          3D Objects
06/25/2024  09:17 AM    <DIR>          advene3
03/26/2024  11:20 AM    <DIR>          Contacts
12/06/2024  09:33 AM    <DIR>          Desktop
12/06/2024  10:19 AM    <DIR>          Documents
12/06/2024  09:56 AM    <DIR>          Downloads
03/26/2024  11:20 AM    <DIR>          Favorites
06/14/2024  09:30 AM    <DIR>          Lightworks
03/26/2024  11:20 AM    <DIR>          Links
12/06/2024  09:33 AM                74 mercurial.ini
05/07/2024  06:07 AM    <DIR>          Music
06/14/2024  10:36 AM    <DIR>          OneDrive - UCSF
12/06/2024  02:34 PM    <DIR>          Pictures
03/26/2024  11:20 AM    <DIR>          Saved Games
03/26/2024  11:20 AM    <DIR>          Searches
03/26/2024  11:20 AM    <DIR>          Videos
07/23/2024  12:21 PM    <DIR>          vimfiles
11/02/2024  01:17 PM    <DIR>          Zotero
10/09/2024  12:04 PM             1,593 _viminfo
               5 File(s)          4,669 bytes
              33 Dir(s)  22,228,291,584 bytes free

C:\Users\spurcell>
```

I can see the folder ‘downloads’ in the directory, which means that I can move to that folder in the Command Prompt. To do this, we will need to use a different command, ‘cd’ or change directory.

So type in

```
cd downloads
```

The Command Prompt processes this function by first reading the command—‘cd’—and then reading the additional information designating specifics about how to execute that command. Typing in ‘cd downloads’ is telling the command prompt to change directories to a directory named ‘downloads’.

Once we’re in downloads we will need to access the directory that holds the files we’re going to use. This directory, which needs to be unzipped, is titled ‘commandline_examples’. To access it type:

```
cd commandline examples
```

You just ran into a problem, right? It is important to know that when navigating folders, you may run into issues with files with spaces—' '—in them. The reason for this error is that the Command Prompt reads each space as a new kind of information (we'll see how this operates later in this lesson!). To fix this, we have to escape out—or create an argument in the Command Prompt—that it recognizes as a single unit, using quotation marks.

Try this instead:

```
cd "commandline examples"
```

There are a few more things to know before we move forward. First, we can move backwards in the directory hierarchy using the following command

```
cd ..
```

This command tells us to go to the directory that contains our current working directory. Let's do the 'cd ..' a second time to return to the user directory.

We can also navigate multiple steps through directories by compounding them using the backslash.

From the user directory (the directory which we started this exercise) type:

```
cd "downloads\commandline examples"
```

The last thing we need to do, before actually using the Command Prompt to do some text analysis is that we'll need a directory into which our analyses will be output. To do this type

```
mkdir outputs
```

This command makes a directory—'mkdir' is short for make directory—and titles the directory whatever we write after the space.

We can navigate to that directory by typing 'cd outputs' or navigate out of it by typing cd ..

Section 2b.3: Why is this Helpful?

[[DO I NEED THIS?]]

Section 3a: Doing Basic Text Analysis Using the Command Line on Mac

Section 3a.1: Doing basic text analysis using the Command Line on Mac

Section 3a.2:

Section 3a 3:

Section 3b: Doing Basic Text Analysis Using the Command Line on Windows

Required: You will need to download and unzip the command prompt files for this lesson.

Section 3b.1: The find command

The command line has the ability to process text files in important and unique ways. While some programs and approaches can do more granular analysis, the command line can introduce us to some simple ideas around computational text analysis.

This also can teach us some of the basics of inputting commands into the command line, and understanding how it functions.

To learn these tools, first navigate to the command prompt folder in your downloads. (If you are confused about how to navigate see section 2b.2.)

Once you are there, let's see what we're working with. To look at what is in the directory type the following code

```
dir
```

```
 Directory of C:\Users\spurcell\Downloads\command prompt

12/10/2024  02:44 PM    <DIR>          .
12/10/2024  02:44 PM    <DIR>          ..
09/05/2024  03:19 PM            18,400 AmericanFederat_TuberculosisDontGiveConsu_1918.txt
09/05/2024  03:19 PM         1,174,872 AnnualReportoftheCanadian17_1917.txt
09/05/2024  03:20 PM           676,151 BrownSampson_IntestinalTuberculosisIts_1926.txt
09/05/2024  03:20 PM           375,119 Burton-Fanning_TheOpen-AirTreatmentofPul_1905.txt
09/05/2024  03:19 PM         1,427,666 Churchill_ConsumptionandTuberculosi_1875.txt
09/05/2024  03:19 PM            84,948 DepartmentofHea_HandBookofHelpforPersonsS_1915.txt
09/05/2024  03:19 PM            25,129 Hayhurst_ConsumptionandPreventable_OhioStateBoardofHealthMon_1913.txt
09/05/2024  03:19 PM           338,368 Heron_EvidencesoftheCommunicabi_1890.txt
09/05/2024  03:20 PM           369,307 Keyes_TheRenewalofLifeArguments_1909-1910.txt
09/05/2024  03:20 PM           456,713 Latham_TheDiagnosisandModernTrea_1907.txt
09/05/2024  03:19 PM         1,436,820 Mayer_ClinicalApplicationofSunl_1925.txt
09/05/2024  03:20 PM           253,341 McLean_TuberculosisAPrimerandPhi_1922.txt
09/05/2024  03:19 PM            34,113 NationalTubercu_StandardsfortheDiagnosisC_1922.txt
09/05/2024  03:20 PM            19,140 ProgrammeofConcertGivenin_1914.txt
09/05/2024  03:20 PM           594,674 Ransome_ACampaignagainstConsumpti_1915.txt
09/05/2024  03:19 PM           944,286 ReportoftheHenryPhippsIns2_1906.txt
09/05/2024  03:19 PM           116,699 Riley_ReportofFiftyCasesIllustr_1897.txt
09/05/2024  03:20 PM           225,366 Ruddock_OnConsumptionandTuberculo_1873.txt
09/05/2024  03:19 PM           361,217 Shuldham_CoughsandTheirCurewithSpe_1878.txt
09/05/2024  03:20 PM            86,969 Smith_CertainAspectsofNaturalan_1916.txt
09/05/2024  03:19 PM         2,058,841 TheAmericanReviewofTuberc11_1925.txt
09/05/2024  03:20 PM           830,728 TheBritishJournalofTuberc5_1911.txt
09/05/2024  03:20 PM         1,705,146 TransactionsoftheFifteent15_1919.txt
09/05/2024  03:19 PM           511,023 TransactionsoftheSectiono_1923.txt
09/05/2024  03:20 PM            61,744 UnitedStatesGov_RecordofCompletedCasesofT_1917.txt
09/05/2024  03:20 PM         1,046,730 vonRuckvonRuck_StudiesinImmunizationAgai_1916.txt
09/05/2024  03:20 PM           196,523 Washington_WashingtononAssimilationC_1956.txt
09/05/2024  03:19 PM           368,172 WoodheadVarrier-Jones_IndustrialColoniesandVill_1920.txt
              28 File(s)     15,798,205 bytes
               2 Dir(s)  26,839,285,760 bytes free
```

The files in the directory are text files, demarcated with the .txt suffix for each file. Text files are just that: files that only contain textual information. There are other file types you may work with regularly like Adobe's portable document format (.pdf), or Microsoft Word's files (.docx), or compressed image files in the joint photographic experts group file type (.jepg or .jpg). All files have this suffix, and computer programs look at these suffixes to determine whether the file is readable.

The [PLACEHOLDER] files in this folder are all text files for materials related to the history of tuberculosis around 1910 and 1920. They were originally taken from HathiTrust, but they have been formatted in such a manner as to have every page be its own unique line in the text. This is what the file BrownSampson_IntestinalTuberculosisIts_1926.txt looks like when opened using a text editor.

```
## p. (#1) ################################################  ♠
## p. (#2) ################################################  A 4% THE LIBRARY OF THE UNIVERSITY OF CALIFORNIA DAVIS GIFT OF SEYMOUR
## p. (#3) ################################################  SAN FRANCISCO TUBERCULOSIS ASSOCIATION ♠
## p. (#4) ################################################  ♠
## p. (#5) ################################################  (ſhe (Iſrubieau iſ cºurtºration $tubiex INTESTINAL TUBERCULOSIS, ITS I
## p. (#6) ################################################  CoPYRIGHT LEA & FEBIGER 1926 PRINTED IN U. S. A. ♠
## p. (#7) ################################################  TO THE MEMORY OF EDWARD LIVINGSTON TRUDEAU FOUNDER OF THE TRUDEAU SANA
## p. (#8) ################################################  \º … ~~~********* e se ! --.-.-.-.~~ ~ ~ ~ ~ ~ (~~~~" pº $ È Ë § @ § ËßS
## p. i (#9) ###############################################  TO THE MEMORY OF EDWARD LIVINGSTON TRUDEAU FOUNDER OF THE TRUDEAU SANA
## p. ii (#10) #############################################  ♠
## p. iii (#11) ############################################  F () REW ORI). THE present volume, the first one of a small series, re
lue of his purpose, his little laboratory immediately began to grow. The writer can recall the time when any medical laboratory in th
we shall feel that we have to the best of our ability carried out one of the precepts of Hippocrates, the Father of Medicine, that we
## p. iv (#12) #############################################  s ! ♠
## p. v (#13) ##############################################  SAN FRANCISCO IUBERCULOSIS ASSOCIATION PREFA (E. THE frequency with wh
ithout question explains why intestinal tuberculosis has for years received so little attention from students of tuberculosis. The me
his work but also for many suggestions and ♠
## p. vi (#14) #############################################  vi PREFACE criticisms. To Dr. Reuben J. Erickson for his splendid stud
## p. vii (#15) ############################################  CONTENTS. CHAPTER I. THE HISTORY OF INTESTINAL TUBERCULOSIs. The "Dyse
## p. viii (#16) ###########################################  viii CONTENTS Incidence of Healing Healing in Untreated Cases • Eviden
e Bowel Movements in Intestinal Tuberculosis Diarrhea . Constipation 112. 113. 114 115. ♠
## p. ix (#17) #############################################  CONTENTS ix The Temperature in Intestinal Tuberculosis . Gastric Sympt
INAL TUBERCULOSIs CHAPTER XX. DIAGNOSIs of INTESTINAL TUBERCULosis. CLINICAL METHODs. Abdominal Examination 116 116 117 117 121 121 1
## p. x (#18) ##############################################  X CONTENTS CHAPTER XXI. DIAGNOSIS OF INTESTINAL TUBERCULosis (Continue
2 167 168 170. 171 172 172 172. 173 174 179 182 182 183 183 184 185. 186 193 196 200. 203 DIAGNOSIS OF INTESTINAL TUBERCULOSIs. RoKiN
## p. 15 (#19) #############################################  CONTENTS Intestinal Hemorrhage Perforation Fecal Abscess . Stenosis Am
## p. 16 (#20) #############################################  ♠
## p. 17 (#21) #############################################  INTESTINAL TUBERCULOSIS. INTRODUCTION. INTESTINAL tuberculosis is ever
e frequent digestive disturbances that occur in this disease. As a matter of fact in any disease whose average duration, including al
and may recur from time to time until the patient is gathered to his fathers. The main question and one hitherto impossible of accura
## p. 18 (#22) #############################################  18 INTESTINAL TUBERCULOSIS (3) to the poisons absorbed from the intest
```

We won't have to change these files in any way, but it usually useful to get a sense of what is happening inside a document before you start using code on it. This document has been formatted in a particular way, but it is obviously not cleaned. There's a lot of extra punctuation that the optical character recognition (OCR) tool did not read correctly. This is fine for this activity, but might run into problems later down the line.

We can also learn about these tools using Voyant—a web-based text analysis tool. (See our text analysis introduction for more information about Voyant.)

Based on this little information we have, we can use the command line to look for specific terms and count them.

While you are in the command prompt directory type in:

```
find /c "tuberculosis" *
```

You will get this output:

```
C:\Users\spurcell\Downloads\command prompt>find /c "tuberculosis" *

---------- AMERICANFEDERAT_TUBERCULOSISDONTGIVECONSU_1918.TXT: 4

---------- ANNUALREPORTOFTHECANADIAN17_1917.TXT: 228

---------- BROWNSAMPSON_INTESTINALTUBERCULOSISITS_1926.TXT: 178

---------- BURTON-FANNING_THEOPEN-AIRTREATMENTOFPUL_1905.TXT: 116

---------- CHURCHILL_CONSUMPTIONANDTUBERCULOSI_1875.TXT: 124

---------- DEPARTMENTOFHEA_HANDBOOKOFHELPFORPERSONSS_1915.TXT: 31

---------- HAYHURST_CONSUMPTIONANDPREVENTABLE_OHIOSTATEBOARDOFHEALTHMON_1913.TXT: 4

---------- HERON_EVIDENCESOFTHECOMMUNICABI_1890.TXT: 55

---------- KEYES_THERENEWALOFLIFEARGUMENTS_1909-1910.TXT: 73

---------- LATHAM_THEDIAGNOSISANDMODERNTREA_1907.TXT: 136

---------- MAYER_CLINICALAPPLICATIONOFSUNL_1925.TXT: 147

---------- MCLEAN_TUBERCULOSISAPRIMERANDPHI_1922.TXT: 84

---------- NATIONALTUBERCU_STANDARDSFORTHEDIAGNOSISC_1922.TXT: 11

---------- PROGRAMMEOFCONCERTGIVENIN_1914.TXT: 0

---------- RANSOME_ACAMPAIGNAGAINSTCONSUMPTI_1915.TXT: 61

---------- REPORTOFTHEHENRYPHIPPSINS2_1906.TXT: 298

---------- RILEY_REPORTOFFIFTYCASESILLUSTR_1897.TXT: 39

---------- RUDDOCK_ONCONSUMPTIONANDTUBERCULO_1873.TXT: 2

---------- SHULDHAM_COUGHSANDTHEIRCUREWITHSPE_1878.TXT: 1

---------- SMITH_CERTAINASPECTSOFNATURALAN_1916.TXT: 24

---------- THEAMERICANREVIEWOFTUBERC11_1925.TXT: 477

---------- THEBRITISHJOURNALOFTUBERC5_1911.TXT: 205

---------- TRANSACTIONSOFTHEFIFTEENT15_1919.TXT: 343

---------- TRANSACTIONSOFTHESECTIONO_1923.TXT: 36

---------- UNITEDSTATESGOV_RECORDOFCOMPLETEDCASESOFT_1917.TXT: 21

---------- VONRUCKVONRUCK_STUDIESINIMMUNIZATIONAGAI_1916.TXT: 243

---------- WASHINGTON_WASHINGTONONASSIMILATIONC_1956.TXT: 0

---------- WOODHEADVARRIER-JONES_INDUSTRIALCOLONIESANDVILL_1920.TXT: 63

C:\Users\spurcell\Downloads\command prompt>
```

What you have done is use the command "find" using a specific set of commands and characters that have defined what the computer will look for.

The "find" command works in sequential order with a space between each command. The command works like this:

> find [parameters marked with a '/' slash] [the search string enclosed in quotation marks ""] [what files it will look at]
>
> **Important Term - String:** A string is a line of characters which a computer can use for specific programmatic functions.

The parameter used in this command is '/c' which counts each time the string appears in the text. And the asterisk '*' at the end of the command is a wild character—which means that the computer will look for all files in the directory.

> More information
> We could use the asterisk to search for a specific file type by typing "*.txt" at the end of this search. We could also use it to define specific files, so if I wanted to only read the transactionsofthefifteent15_1919.txt and transactionsofthesectiono_1923 I could write "trans*" at the end of my search command.

We can think of a find command as having a kind of grammar and logic the command we used above,

```
find /c "tuberculosis" *
```

Can be read as

```
In the current directory, find and count every
instance of the term "tuberculosis" in every file.
```

There are a few other parameters that are important to know, like the /i which searches for the string without considering capitalization. This can matter because sometimes terms may be capitalized or not capitalized across a document.

```
C:\Users\spurcell\Downloads\command prompt>find /c /i "tuberculosis" *

---------- AMERICANFEDERAT_TUBERCULOSISDONTGIVECONSU_1918.TXT: 5

---------- ANNUALREPORTOFTHECANADIAN17_1917.TXT: 309

---------- BROWNSAMPSON_INTESTINALTUBERCULOSISITS_1926.TXT: 255

---------- BURTON-FANNING_THEOPEN-AIRTREATMENTOFPUL_1905.TXT: 122

---------- CHURCHILL_CONSUMPTIONANDTUBERCULOSI_1875.TXT: 166

---------- DEPARTMENTOFHEA_HANDBOOKOFHELPFORPERSONSS_1915.TXT: 43

---------- HAYHURST_CONSUMPTIONANDPREVENTABLE_OHIOSTATEBOARDOFHEALTHMON_1913.TXT: 7

---------- HERON_EVIDENCESOFTHECOMMUNICABI_1890.TXT: 66

---------- KEYES_THERENEWALOFLIFEARGUMENTS_1909-1910.TXT: 84

---------- LATHAM_THEDIAGNOSISANDMODERNTREA_1907.TXT: 139

---------- MAYER_CLINICALAPPLICATIONOFSUNL_1925.TXT: 189

---------- MCLEAN_TUBERCULOSISAPRIMERANDPHI_1922.TXT: 126

---------- NATIONALTUBERCU_STANDARDSFORTHEDIAGNOSISC_1922.TXT: 16

---------- PROGRAMMEOFCONCERTGIVENIN_1914.TXT: 1

---------- RANSOME_ACAMPAIGNAGAINSTCONSUMPTI_1915.TXT: 99

---------- REPORTOFTHEHENRYPHIPPSINS2_1906.TXT: 332

---------- RILEY_REPORTOFFIFTYCASESILLUSTR_1897.TXT: 47

---------- RUDDOCK_ONCONSUMPTIONANDTUBERCULO_1873.TXT: 18

---------- SHULDHAM_COUGHSANDTHEIRCUREWITHSPE_1878.TXT: 3

---------- SMITH_CERTAINASPECTSOFNATURALAN_1916.TXT: 33

---------- THEAMERICANREVIEWOFTUBERC11_1925.TXT: 552

---------- THEBRITISHJOURNALOFTUBERC5_1911.TXT: 295

---------- TRANSACTIONSOFTHEFIFTEENT15_1919.TXT: 464

---------- TRANSACTIONSOFTHESECTIONO_1923.TXT: 47

---------- UNITEDSTATESGOV_RECORDOFCOMPLETEDCASESOFT_1917.TXT: 25

---------- VONRUCKVONRUCK_STUDIESINIMMUNIZATIONAGAI_1916.TXT: 374

---------- WASHINGTON_WASHINGTONONASSIMILATIONC_1956.TXT: 0

---------- WOODHEADVARRIER-JONES_INDUSTRIALCOLONIESANDVILL_1920.TXT: 69

C:\Users\spurcell\Downloads\command prompt>
```

Before we go to the next step, try searching for a few terms of these terms:

- Google
- E. L. Trudeau
- tuberculin

The first term ‘google’ shouldn’t show up, because these documents come from the early twentieth century. The other two terms E. L. Trudeau and tuberculin refer to specific historical actors and practices. What is important, at least for E. L. Trudeau is that he might show up in

the files in a few different ways, because E. L. might be written out as his full name Edward Livingston Trudeau.

Section 3b.2 Why Find Matters

This simple function can be helpful in parsing large numbers of text files for specific information. The dataset we're using in this exercise comes from a larger collection of books—a corpus—which have information about a range of issues related to tuberculosis around the turn of the twentieth century.

Find helps us parse information based on what we might already know about our project or research question. We can use find to help find texts that may be valuable for our research. For example, in the previous section I asked for you to search tuberculin. Tuberculin was a pharmaceutical developed by Robert Koch to try to cure tuberculosis. Its clinical effectiveness was disproven, but it was something that doctors tested as a potential cure. It was also used as a test for tuberculosis.

What you'll notice if you search for tuberculin that some books cite it quite a bit, and others do not mention it at all. If your research project involved that compound, those trials, or its use as a test for tuberculosis, then you would have a clear idea of what books might be most valuable to read first.

There are some limitations: find only really works for one string at a time, so if you were searching for something that has different names (tuberculosis vs. pthisis vs. consumption for example) you might run into issues finding relevant work. (Although, you can use the '*' to search for terms that begin or end with a specific string of characters—"tubercul*" would search for "tuberculosis" and "tuberculous" and "patholog*" would search for "pathology" "pathological" and so on . . .)

Section 3b.3 findstr

Where Mac computers have a single term that includes all the functionality we're trying to achieve in this activity (called 'grep'), for Windows, the 'find' function is accompanied by another function—'findstr'.

The 'findstr' function has a wider utility than the 'find' function, because it has a lot more and different parameters which it can use. What we are going to use it for is to find the unique pages for each instance of a given string. This will give us information as to the proximity of each term, so that we can produce different results from it.

What we are going to do is search for each line for each page a specific term is used. The reason for this is that we can then use these extracted lines for a kind of simplified text analysis. It also lets us learn a few additional functions.

The same grammar of using 'findstr' applies. So

```
findstr /i "tuberculin" *
```

would read something like

```
Find every line in every document that includes the
word "tuberculin" without considering capitalization.
```

Try running that code.

What you'll realize when you run the command is that it outputs A LOT of text. In the next step we'll learn how to create a new directory and how to output files in the command line.

Section 3b.4 Making Files

In order to address this problem of having a long output in the terminal, we will have to take that output and export it into a new file. We do this because it allows us to have it in a form that we can read using a text reader, and also because it can save the outputs we find.
But first, so it does not effect any other outputs that we're exporting, we should create a new directory. We do this by using the 'mkdir'—make directory—command:

```
mkdir outputs
```

This command will make a directory with the same name as the next word or string.
Type

```
dir
```

And you can see the directory you just made.
To create a file with an command output you will need to use the '>' at the end of the command you enter.
Let's make a new folder counting the number times 'tuberculin' is used:

```
find /c /i "tuberculin" *.txt >
outputs\tuberculinfind.txt
```

This command looks a bit different than the others, so let's break it down.

```
find /c /i "tuberculin"
```

This part of the code is the same as before. We're counting "tuberculin" as it appears, without considering different capitalization.

```
*.txt
```

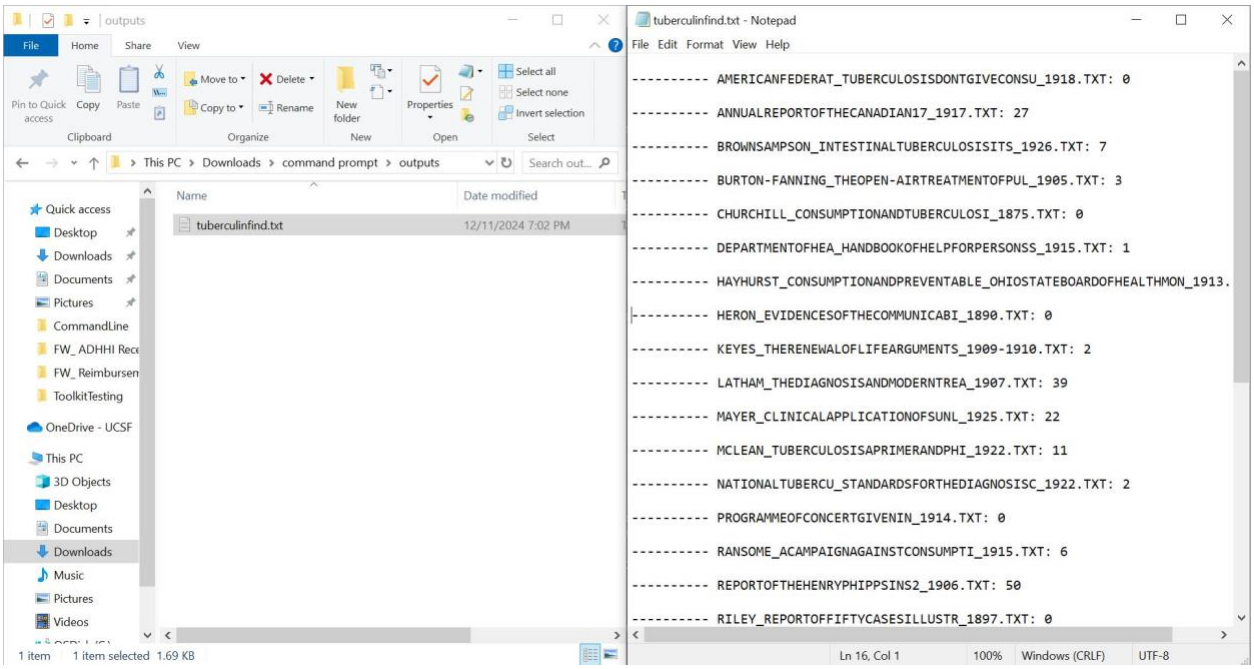
This is different because we have a different object in our directory: the outputs folder. We don't want the code to look at that, so the *.txt is basically saying: look at every file with a '.txt' suffix and ignore any other files.

```
> outputs\tuberculinfind.txt
```

This is the command that is telling the command prompt to export the command output into a new file. (Or if that file exists to overwrite the file.) The '>' tells the command prompt to do this and then the entry that follows is a folder hierarchy and a file name. So 'outputs\tuberculinfind.txt' is telling the command prompt to look into a directory inside the current directory named 'outputs' and then make a file with the name 'tuberculinfind.txt'.
Now run that command:

```
find /c /i "tuberculin" *.txt >
outputs\tuberculinfind.txt
```

You should have in that outputs directory a new file named tuberculinfind.txt and it should look like this:



Now try to do the same with the findstr command.

```
findstr /i "tuberculin" *.txt >
outputs\tuberculinstring.txt
```

Remember that we want to make a new file, so it'll have to have a new name. What you have now is a collection of pages that describe tuberculin.

Section 4: Synthesis

Using the tools you have learned in this lesson, I want you to create a new file. I want you to search for terms related to meningeal tuberculosis, using "meningeal" as the search term—meningeal, meningitis, meninges.

Remember: you do not have to search each of these three terms separately. There is a function we learned in this lesson that will help you search multiple words with a similar beginning or ending.

Spoiler: Meningeal Tuberculosis Search (Windows}

findstr /i "mening*" *.txt > outputs\meningealsearch.txt

What can we do with this text file?? We counted terms earlier in this lesson (see sections 3b.1 and [!!!mac version]), and that helped us find specific texts that focused on specific interests in the period. But now we are generating texts, which are themselves too long to gloss.

What we can do with these files is use them for **textual analysis.**

> **Important Term - Textual Analysis**: Textual analysis (or text analysis for short) refers to any method that examines large amounts of text using computational tools. Text analysis historically was used to count the use of specific words, and look at their context with other words. More contemporary text analysis uses large language models and machine learning to develop more complex groupings of interrelated words (topic

modelling) or weight texts based on the positive or negative affect of words used (sentiment analysis).

There are more lessons on text analysis elsewhere in this toolkit ([!!!!links go here]), but wec an look at the documents we generated and see that they are covering very different parts of the **corpus** we examined. For this lesson, we will use Voyant: an open-source text analysis platform that is very well suited for teaching.

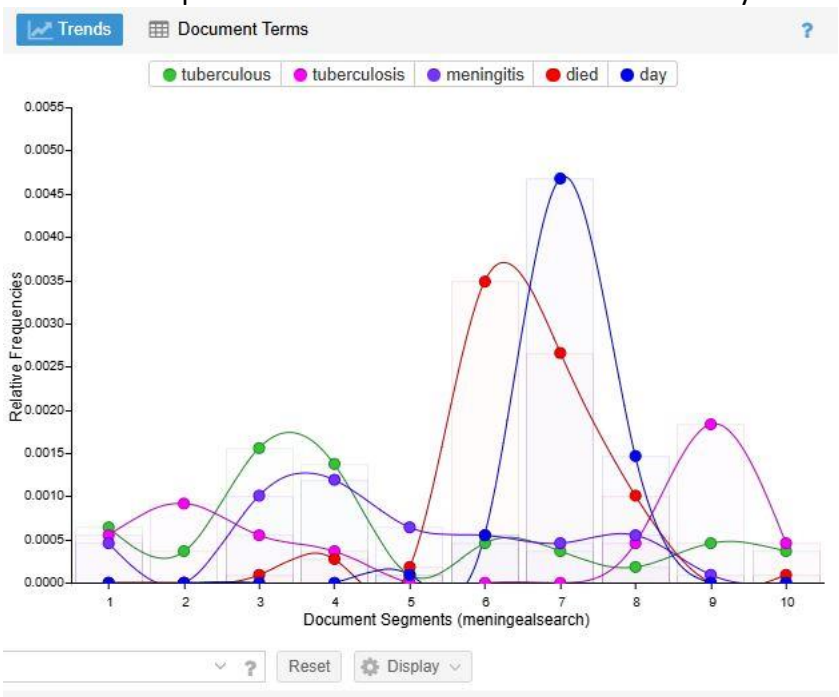
Go to voyant-tools.org in a separate tab in your browser.

Once there, click on the "upload" button and navigate to the text you generated. Once you upload it, you'll see a dashboard with LOTS of different pieces of information.

We will cover some of the various methods that Voyant enables elsewhere in this toolkit ([!!!Links go here]), but for now I want us to focus on one interface.

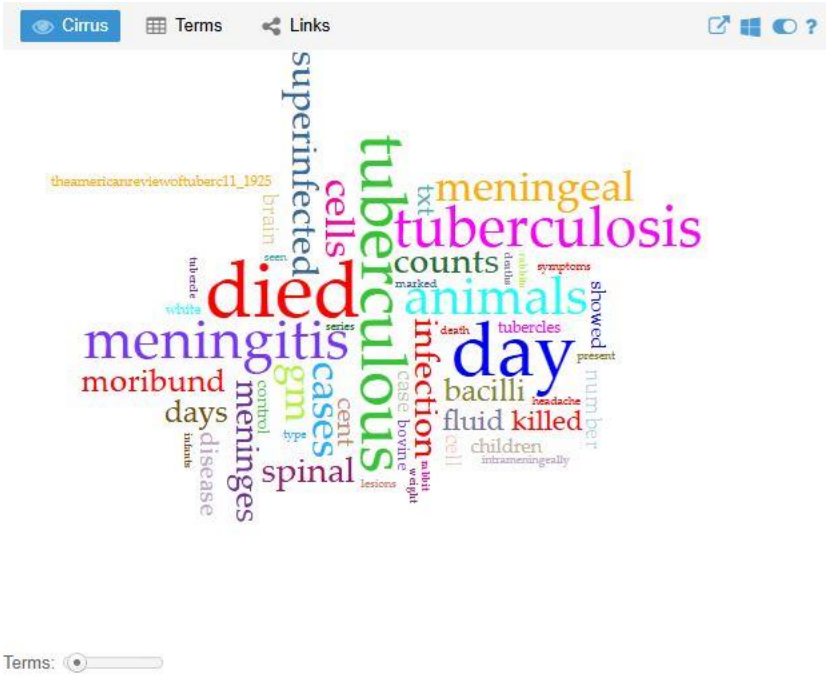
In the upper right-hand corner of the dashboard there is a panel that shows the frequency of terms across the document. Voyant breaks these terms down when it processes the file. If you have multiple files, it will show every file in the collection in this view, or if you have one file, like in this example, it will partition that file evenly. It also generates the relative frequency based on the size of the chunk it generates. This view can help point to specific places in the file or collection of files where interests are unique.

Look at the Voyant generated graphs. Here is the meningeal tuberculosis graph. What is going on with that spike in the use of the terms "died" and "day"?



By looking at this interface we may be to make some preliminary assumptions: maybe "died" and "day" are connected because the use meningeal is used in diagnosis, and may be mentioned in case studies and case histories. "Died" and "day" may be referring to moments when patients died, or days when they had notable symptoms.

Let's look quickly at the word cloud in the upper left-hand corner of the interface:



Word clouds are generated by counting every instance of every unique term in the file, and then scaling the size of each word based on its relative frequency to the others. We can see the five terms in the previous graph, but a bunch of other terms that are not quite as significant. There are a few things I want to point out. First, is that there's some words from the generated file: "theamericanreviewoftuberc11_1925" and "txt". These occurred because we did not **clean** the file before uploading it.

> **Important Concept: Data Cleaning** – Often we have to make the files we are using in our digital research compatible with our platforms. We have to deal with any errant mistakes or issues with those data sources. Removing the unnecessary information in data files is known as data cleaning.

Second, there are some interesting terms popping up: "moribund", "tubercles", "spinal", "fluid", "symptoms". These could be used to do further searches, or could help determine something about the relationship of the word meningeal and this kind of symptomatic language.

Importantly, the search we did looks different than if we uploaded the entire collection of texts to Voyant.

These are much more general because they describe the entire corpus. Searching and generating files from those searches can make more specific explorations of the broader collection of texts.